Lab 5: Static Code Analysis

Voting and Election Management System

Name: Kallepalli Sai Meghana

SRN: PES1UG24AM911

Section: E

File analysed: election_system.py

1. Objective

The aim of this lab was to improve the quality, security and style of a Python program using three static analysis tools: Pylint, Bandit and Flake8. Each tool was run on the assigned script, the reported issues were documented and fixed, and the tools were run again to confirm the fixes.

2. Tools Used

Pylint checks code quality and logical problems such as unused variables, naming conventions, missing docstrings and risky patterns, and gives the code a score out of 10. Bandit is a security scanner that looks for dangerous functions and insecure coding patterns and rates each finding by severity and confidence. Flake8 enforces PEP 8 style, covering formatting, whitespace, line length and basic errors such as unused imports. All three were installed in a Python virtual environment (Python 3.11.9 on Windows).

3. Running the Original Program

Before running any tools, I ran the program to see what it actually does. Almost every line of output was wrong: an 18 year old could not register, a plain voter was treated as an election officer, Ben was allowed to vote twice, a vote at 18:30 was accepted after polls closed, 'bob' was counted separately from 'Bob', turnout came out at 166 percent, and all three announcements printed 'NOTA: 0'.

```
(venv) PS C:\Users\meghu\OneDrive\Desktop\L5_62_voting-election-system> python election_system.py
Register Asha (age 18): False
Is a plain 'voter' an election officer? True
Ben votes Alice: True
Ben votes Alice again: True
Chen votes 'bob': True
Dia votes Bob at 18:30: True
Postal ballots imported: 1
Tally: {'Alice': 2, 'bob': 1, 'Bob': 2}
Winner: Alice
Alice %: 40
Turnout %: 166.66666666666669
Announcements: ['NOTA: 0', 'NOTA: 0', 'NOTA: 0']
Receipt: RCPT-958280
```

Figure 1: Output of the original program

4. Initial Analysis

The three tools were run on the original file and the output was saved to before/pylint_report.txt, before/bandit_report.txt and before/flake8_report.txt.

Tool	Result	Main findings
Pylint	5.33/10, 38 messages	Bidirectional Unicode, naming, missing docstrings, unused imports and variables, cell variable in loop, bare except
Bandit	7 issues: High 2, Medium 1, Low 4	Trojan Source (B613), MD5 (B324), unsafe XML (B319, B409), random (B311), hardcoded password (B105), try/except/pass (B110)
Flake8	30 warnings	E302 blank lines, E501 long lines, W291 trailing whitespace, F401 unused imports, F841 unused variables, E722 bare except

While generating the Bandit report on Windows, Bandit crashed with a UnicodeEncodeError. The report contained the hidden U+202E character from line 22, and the default Windows encoding (cp1252) could not write it. Setting PYTHONUTF8=1 before running the command fixed this. It was a useful reminder that the hidden character was invisible in the editor but still real enough to break a tool.

5. Issues Identified

Line numbers refer to the original file. Issues are listed with the highest severity first.

Issue	Tool / Code	Severity	Line(s)	Description	Fix
Trojan Source bidi characters	Bandit B613, Pylint E2502	High	22	Hidden right-to-left Unicode characters in isOfficer made the check look correct, but it returned True for 'voter'.	Deleted the line and retyped it as return role == "officer".
Weak MD5 hash	Bandit B324	High	17	MD5 is broken and was used to hash voter IDs.	Replaced with hashlib.sha256.
Unsafe XML parsing	Bandit B319, B409	Medium	7, 75	xml.dom.pulldom is open to entity expansion attacks on untrusted postal ballot XML.	Switched to defusedxml.pulldom, which rejects entity declarations.
try / except / pass	Bandit B110, Pylint W0702, Flake8 E722	Low	82-83	Bare except silently swallowed every error.	Catch only DefusedXmlException and ExpatError and raise a clear ValueError.
Predictable receipts	Bandit B311	Low	72	random is not cryptographically secure, so receipt numbers could be guessed.	Used secrets.randbelow.
Hardcoded password	Bandit B105	Low	9	ADMIN_PASSWORD stored in plain text and never used.	Removed the variable.
Late-binding lambda	Pylint W0640	Bug	68	All lambdas captured the last loop value, so every announcement read 'NOTA: 0'.	Replaced with an f-string list comprehension.

Issue	Tool / Code	Severity	Line(s)	Description	Fix
Unused imports and variables	Pylint W0611/W0612, Flake8 F401/F841	Style	1-3, 32, 45, 55	os, sys, json, receipt, total and now were never used.	Removed them; the receipt is now stored on the voter record.
Shadowed global	Pylint W0621	Style	58	countFor's parameter ballots hid the global list.	Renamed the parameter to ballot_list.
Naming and docstrings	Pylint C0103, C0114, C0116	Style	Whole file	camelCase function names and no docstrings.	Renamed to snake_case and added docstrings.
PEP 8 layout	Flake8 E302, E305, E501, W291	Style	Whole file	One blank line between functions, lines over 79 characters, trailing spaces.	Two blank lines around functions, long lines split, trailing spaces removed.

6. Fixes Applied

The fixes were made in three commits, one for each tool.

Commit 1: Fix Bandit security issues (b281098). The most important fix was in isOfficer. The comparison string contained right-to-left override characters, a technique known as Trojan Source, so the code displayed as a check for election officers while actually returning True for almost any role. The line was deleted and retyped as return role == "officer". MD5 was replaced with SHA-256, random was replaced with secrets for receipt numbers, the XML parser was changed to defusedxml.pulldom, the unused hardcoded password was removed, and the bare try/except/pass was replaced with handling of specific exceptions. The defusedxml version of pulldom does not provide the START_ELEMENT constant, so a local constant with the same value was added. After this commit Bandit reported no issues and the officer check printed False.

```
(venv) PS C:\Users\meghu\OneDrive\Desktop\L5_62_voting-election-system> python election_system.py
Register Asha (age 18): False
Is a plain 'voter' an election officer? False
Ben votes Alice: True
Ben votes Alice again: True
Chen votes 'bob': True
Dia votes Bob at 18:30: True
Postal ballots imported: 1
Tally: {'Alice': 2, 'bob': 1, 'Bob': 2}
Winner: Alice
Alice %: 40
Turnout %: 166.66666666666669
Announcements: ['NOTA: 0', 'NOTA: 0', 'NOTA: 0']
Receipt: RCPT-371687
```

Figure 2: Output after commit 1 (officer check fixed)

Commit 2: Fix Pylint issues and logic bugs (ba0dc60). Functions were renamed to snake_case, docstrings were added, unused imports and variables were removed, the shadowed parameter was renamed, and an if __name__ == "__main__" guard was added. The late-binding lambda in announce() was replaced with a list comprehension. This commit also fixed the logic bugs found by running the program, which none of the tools reported. A shared record_ballot function now applies the same eligibility rules to both in-person and postal votes. Pylint went from 5.33 to 10.00.

Bug (not flagged by any tool)	Before	After	Fix
Voters aged exactly 18 rejected	Asha (18): False	True	age > 18 changed to age >= 18
Double voting allowed	Ben votes again: True	False	Check the voter's voted flag before recording
Votes after polls close	Vote at 18:30: True	False	when.hour > 18 changed to >= 18
Candidate names not validated	'bob' and 'Bob' counted separately	Merged into Bob	Case-insensitive match against the candidate list
Turnout over 100%	166.67%	75.0%	Count registered voters who voted, not raw ballots
Postal ballots from anyone	Voter attribute ignored	Eligibility checked	Postal ballots go through the same record_ballot check
Divide by zero on empty election	Crash	Returns 0.0	Guard for empty ballots and voters
Ties	First candidate silently wins	Returns None	Detect multiple leaders

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 5.33/10, +4.67)

(venv) PS C:\Users\meghu\OneDrive\Desktop\L5_62_voting-election-system> pylint election_system.py

--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

(venv) PS C:\Users\meghu\OneDrive\Desktop\L5_62_voting-election-system> python election_system.py
Register Asha (age 18): True
Is a plain 'voter' an election officer? False
Ben votes Alice: True
Ben votes Alice again: False
Chen votes 'bob': True
Dia votes Bob at 18:30: False
Postal ballots imported: 1
Tally: {'Alice': 1, 'Bob': 2, 'NOTA': 0}
Winner: Bob
Alice %: 33.33
Bob votes counted directly: 2
Turnout %: 75.0
Announcements: ['Alice: 1', 'Bob: 2', 'NOTA: 0']
Receipt: RCPT-712358
(venv) PS C:\Users\meghu\OneDrive\Desktop\L5_62_voting-election-system>
```

Figure 3: Pylint score of 10.00/10 and corrected program output after commit 2

Commit 3: Fix Flake8 style issues and add after reports (b850656). Two blank lines were added around every top-level function and before the main guard, long lines were split to stay under 79 characters, and trailing whitespace was removed. The after reports were generated and saved in the after folder. Flake8 now produces an empty report.

```
(venv) PS C:\Users\meghu\OneDrive\Desktop\L5_62_voting-election-system> bandit -r election_system.py
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.11.9
Run started:2026-10-28 13:21:46.916415+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 89
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
(venv) PS C:\Users\meghu\OneDrive\Desktop\L5_62_voting-election-system>
```

Figure 4: Bandit after all fixes, no issues identified

```
(venv) PS C:\Users\meghu\OneDrive\Desktop\L5_62_voting-election-system> python election_system.py
Register Asha (age 18): True
Is a plain 'voter' an election officer? False
Ben votes Alice: True
Ben votes Alice again: False
Chen votes 'bob': True
Dia votes Bob at 18:30: False
Postal ballots imported: 1
Tally: {'Alice': 1, 'Bob': 2, 'NOTA': 0}
Winner: Bob
Alice %: 33.33
Bob votes counted directly: 2
Turnout %: 75.0
Announcements: ['Alice: 1', 'Bob: 2', 'NOTA: 0']
Receipt: RCPT-489822
(venv) PS C:\Users\meghu\OneDrive\Desktop\L5_62_voting-election-system>
```

Figure 5: Final program output after commit 3

7. Final Results

Tool	Before	After
Pylint	5.33/10 (38 messages)	10.00/10
Bandit	7 issues (High 2, Medium 1, Low 4)	No issues identified
Flake8	30 warnings	0 warnings

The program still does its job and now gives correct results: Asha can register, the repeat vote and the late vote are rejected, 'bob' is counted for Bob, Bob wins with 2 votes, Alice has 33.33 percent, turnout is 75 percent and the announcements show the real counts.

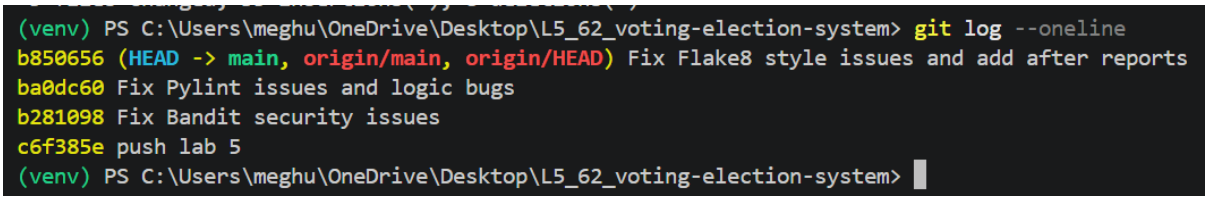


Figure 6: Git log showing the three fix commits

8. Reflection

Which issues were the easiest to fix, and which were the hardest? The easiest were the Flake8 layout warnings and the Pylint naming and docstring messages, since they only needed blank lines, line splits and renames. The hardest was the isOfficer check, because the line looked correct and only Bandit and Pylint revealed the hidden characters. Moving to defusedxml was also tricky because its pulldom module does not expose START_ELEMENT, so the first attempt failed at runtime even though Bandit was satisfied. The logic bugs took the most thought because no tool pointed them out.

Did the tools report any false positives? Not really. The closest was Bandit B105, which flags any variable named like a password that holds a string, regardless of whether it is a real secret. Here it was a genuine hardcoded password. B311 on random would be a false positive in a game, but not for receipts that should be unguessable. The bigger problem was false negatives: none of the tools caught the age check, double voting, late votes or turnout above 100 percent.

How would you integrate static analysis into your development workflow? I would run Flake8 and Bandit in a pre-commit hook so style and security problems are caught before code is committed, and run all three tools in CI on every pull request. The build would fail on any Medium or High Bandit finding or if the Pylint score drops below 9.5. Unit tests would run alongside, since the tools cannot check whether the program gives correct results.

What improvements did you observe after applying the fixes? Pylint improved from 5.33 to 10.00, Bandit went from 7 issues to none and Flake8 from 30 warnings to none. The code is easier to read with consistent names and docstrings, it no longer crashes on an empty election, it rejects malicious XML, and most importantly the results it prints are now correct.

9. Conclusion

Static analysis found real security problems, including a hidden-character backdoor that was impossible to spot by reading the code, and brought the file to a clean state on all three tools. However, the tools only check how code is written, not what it does. Several serious bugs were found only by running the program and checking its output, so static analysis works best combined with testing and careful review.